

Efficient Memory Management for LLMs via Geometric Token Pruning

A Resource-Constrained Inference Simulation

Aditya Pratap Singh Bhadoria (24114005)
Deokar Parth Rajesh (24114032)

Course: Fundamentals of AI/ML (CSC-351)

Abstract

As Large Language Models (LLMs) process increasingly longer contexts, the memory footprint required to store historical token states scales linearly[cite: 8, 10]. This poses a severe bottleneck for edge deployment where hardware resources are heavily constrained[cite: 8]. This report evaluates a training-free memory eviction algorithm that analyzes the geometric properties of token embeddings to identify and remove redundant information[cite: 8, 10]. Through a synthetic information retrieval experiment, we demonstrate that angular proximity filtering can achieve 100% data retention under a 75% compression constraint, significantly outperforming standard sliding-window approaches[cite: 10].

1 Introduction

During autoregressive generation, transformers utilize a memory buffer (Key-Value cache) to store previously computed states, bypassing redundant calculations[cite: 8, 10]. While highly effective for accelerating inference, this buffer grows continuously as the context window expands[cite: 10]. For resource-constrained environments, managing this buffer necessitates imposing a strict memory ceiling[cite: 8]. Traditional methods default to a First-In-First-Out (FIFO) or sliding-window policy, discarding the oldest chronological data[cite: 10]. However, critical instructions or facts are often located early in a prompt; naive deletion frequently destroys this vital contextual information[cite: 10]. We propose and evaluate an algorithm that preserves tokens based on their informational uniqueness rather than their position in the sequence[cite: 10].

2 Proposed Methodology

Recent literature indicates a correlation between a token’s geometric distinctiveness and its ultimate importance[cite: 8]. Tokens that cluster tightly together in the embedding space carry redundant semantic weight, whereas outliers often contain rare, specific information required for reasoning or retrieval[cite: 10].

Our implemented algorithm, termed *Geometric Token Pruning*, operates directly on the memory vectors without requiring computationally expensive full-attention materialization[cite: 8, 10]. The eviction policy operates efficiently via the following steps[cite: 10]:

1. **Centroid Calculation:** Compute the mean vector representing the semantic center of the current memory pool:

$$\mu = \frac{1}{n} \sum_{i=1}^n k_i \quad (1)$$

2. **Divergence Scoring:** Calculate the angular proximity (cosine similarity) of each stored token to this calculated centroid[cite: 8, 10].

$$\text{Score}(k_i) = -\cos(k_i, \mu) \quad (2)$$

3. **Redundancy Eviction:** Tokens that are highly similar to the centroid are flagged as repetitive data[cite: 10]. Distinct outliers are retained, while the tightly clustered, redundant tokens are safely evicted to satisfy memory constraints[cite: 10].

3 Experimental Setup

To isolate and stress-test the eviction logic efficiently, we developed a synthetic “Needle-in-a-Haystack” retrieval simulation[cite: 10].

- **Data Generation:** We initialized a sequence of 256 token embeddings (dimension $d = 48$) acting as correlated distractor keys[cite: 10]. A highly distinctive “target” vector carrying a specific integer answer was inserted into this sequence[cite: 10].
- **Compression Constraint:** The sequence was processed in block sizes of 16[cite: 10]. A strict cache budget of 64 tokens was enforced, representing a 75% reduction in cache size[cite: 10].
- **Evaluation:** Following eviction, a query vector highly correlated with the target was used to execute a retrieval search[cite: 10]. The experiment was averaged over 100 independent trials with a fixed random seed (20260823)[cite: 10].

4 Results and Discussion

The experimental results demonstrate the superiority of geometric filtering over standard chronological eviction[cite: 10]. Table 1 summarizes the retention accuracy and computational overhead across the tested policies.

Table 1: Retrieval Accuracy and Runtime Comparison

Method	Cache Size	Target Retention	Accuracy	Runtime (ms)
Unrestricted (Full)	256	100%	100%	0.272
Keep-Recent (FIFO)	64	0%	0%	0.308
Geometric Pruning	64	100%	100%	1.026

- **Unrestricted Baseline:** Processing the full 256-vector sequence without memory limits yielded perfect retrieval accuracy at a baseline latency of 0.272 ms[cite: 10].
- **Sliding Window (Keep-Recent):** Constraining the memory to 64 tokens completely failed to retain the crucial early context[cite: 10]. The target vector was consistently overwritten, resulting in 0% retrieval accuracy[cite: 10].
- **Geometric Token Pruning:** Our methodology retained the target vector across all 100 trials, maintaining 100% accuracy despite the severe 75% memory bottleneck[cite: 10]. The computational trade-off was a minor latency increase to 1.026 ms due to the underlying cosine similarity math, providing an acceptable trade-off for perfect context retention[cite: 10].

5 Conclusion

This report successfully implements and validates a mathematically grounded, training-free approach to sequence compression[cite: 8]. By evaluating the angular proximity of features to their local centroid, we can safely discard up to 75% of a context window without losing isolated, critical facts[cite: 10]. While slight computational overhead is introduced, the capability to run robust sequence retrieval within a strict hardware memory budget makes this an ideal architecture for edge AI applications[cite: 8, 10].